

JOBHUNTMANAGER

# フロントエンド完全解説書

React + TypeScript + Vite + Tailwind CSS

初心者から初級者向け  
実装コードとデータフローで学ぶ  
面接で説明できるフロントエンド設計

対象: frontend配下の実装・設定・テスト

## この本の読み方

このアプリのフロントエンドは、次の順番で読むと理解しやすい。

1. `main.tsx`でReactが起動する
2. `App.tsx`でRouterと認証Providerを組み合わせる
3. `router.tsx`でURLと画面の対応を決める
4. `AuthProvider.tsx`でログイン状態を全体管理する
5. `client.ts`でRails APIとの通信方法を共通化する
6. `KanbanPage.tsx`と`ApplicationDetailPage.tsx`で画面を構成する
7. `features`配下で機能ごとのAPI・型・UIを管理する

本書のコード例は、理解に必要な部分を抜粋している。コード全体を暗記する必要はない。重要なのは、次の3点を説明できることである。

- ・ そのコードは何を担当しているか
- ・ なぜそのファイルに置かれているか
- ・ ユーザー操作から画面更新まで、どの順番で処理されるか

### 図の読み方

```
graph TD; A[ユーザー操作] --> B[React Component]; B --> C[APIモジュール]; C --> D[Axios client]; D --> E[Rails API];
```

上から下へ処理が進み、Railsから返されたデータをStateへ保存すると、Reactが画面を再描画する。

## 目次

1. フロントエンド全体構成
2. React基礎
3. TypeScript
4. React Router
5. 認証機能
6. AxiosとAPI通信
7. カンバン画面
8. 応募詳細画面
9. エラーハンドリング
10. 面接で聞かれそうな質問
11. 付録A: 全ファイル役割一覧
12. 付録B: API対応表
13. 付録C: 学習用語集と確認問題

# 第1章 フロントエンド全体構成

## 1.1 このフロントエンドの役割

JOBHUNTMANAGERは、RailsがHTMLを作る構成ではない。RailsはJSON APIを提供し、画面はReactが作る。

### 図解

1. React SPA -- Axios + JSON --> Rails API
2. Rails API → PostgreSQL
3. Rails API -- JSON Response --> React SPA
4. React SPA -- State更新・再描画 --> ユーザー

React側が担当するもの：

- URLに応じた画面表示
- ログイン状態の保持
- フォーム入力
- API呼び出し
- ローディングやエラー表示
- カンバン・面接・タスク・メモの操作
- Tailwind CSSによる見た目

Rails側が担当するもの：

- 認証とJWT発行
- データの保存
- バリデーション
- 所有権の確認
- 他ユーザーのデータへのアクセス防止
- JSONレスポンス

重要なのは、フロントエンドの入力チェックだけを信用しないことである。Reactでボタンを無効化しても、APIを直接呼ばれる可能性があるため、最終的な安全性はRails側でも保証する。

## 1.2 使用技術

| 技術              | このアプリでの役割                  |
|-----------------|----------------------------|
| React           | Componentを組み合わせて画面を作る      |
| TypeScript      | APIデータ、Props、Stateの型を定義する  |
| Vite            | 開発サーバー、TypeScriptビルド、成果物生成 |
| Tailwind CSS v4 | classNameでスタイルを指定する        |
| React Router    | URLと画面を対応させる               |
| Axios           | Rails APIへHTTPリクエストを送る     |
| Vitest          | APIモジュールの単体テスト             |
| ESLint          | コード上の問題を静的に検出する            |

## 1.3 ディレクトリ構成

主要部分を抜き出すと、次の構成になっている。

```
frontend/
├── src/
│   ├── app/
│   │   ├── layouts/
│   │   ├── providers/
│   │   └── router.tsx
│   ├── features/
│   │   ├── applications/
│   │   ├── auth/
│   │   ├── companies/
│   │   ├── interviews/
│   │   ├── jobPostings/
│   │   ├── kanban/
│   │   ├── notes/
│   │   └── tasks/
│   ├── pages/
│   ├── shared/
│   │   ├── api/
│   │   ├── components/
│   │   ├── routes/
│   │   ├── types/
│   │   └── utils/
│   ├── App.tsx
│   ├── index.css
│   └── main.tsx
├── package.json
├── tsconfig.app.json
├── eslint.config.js
└── vite.config.ts
```

### app

アプリ全体に関する構成を置く。

- ・ 全体ルーティング
- ・ 共通レイアウト
- ・ 認証ContextのProvider

### features

機能単位でコードをまとめる。

```
features/tasks/
├── api/tasksApi.ts
├── components/
│   ├── TaskSection.tsx
│   ├── TaskForm.tsx
│   └── TaskList.tsx
└── types.ts
```

この構成により、「タスク機能を修正したい」ときに関連ファイルを見つけやすい。

### pages

URLと直接対応する画面を置く。

- ・ [LoginPage](#)
- ・ [SignupPage](#)
- ・ [KanbanPage](#)
- ・ [ApplicationDetailPage](#)
- ・ [NotFoundPage](#)

### shared

複数機能から利用する共通処理を置く。

- ・ Axios client
- ・ APIエラー変換
- ・ ローディング表示
- ・ 認証ルート
- ・ 日付処理

## 1.4 なぜこの構成にしたのか

小規模なアプリなら、すべてを `components` へ入れても動く。しかし機能が増えると、どのAPIや型が何のためのものか分かりにくくなる。

JOBHUNTMANAGERでは、次の責務分離を採用している。

| 層         | 主な責務                |
|-----------|---------------------|
| Page      | 画面全体のデータ取得・レイアウト    |
| Section   | 1機能の状態管理とCRUD       |
| Form      | 入力値・送信状態・項目別エラー     |
| List/Card | データの表示と操作イベント       |
| API       | URL、HTTPメソッド、JSON構造 |
| types     | APIデータとフォーム値の型      |

これにより、UIと通信処理が密結合になりにくい。

## 1.5 アプリ起動の流れ

### main.tsx

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```

`index.html`にある`<div id="root"></div>`へReactアプリを描画する。

!は「ここでは`null`ではない」とTypeScriptへ伝える記号である。`index.html`に必ず`root`がある前提なので使用できる。

`StrictMode`は開発中に副作用の問題を見つけやすくする。開発環境ではEffectが追加で実行されることがあるため、API取得処理には後述する`cleanup`が必要になる。

### App.tsx

```
<BrowserRouter>  
  <AuthProvider>  
    <AppRouter />  
  </AuthProvider>  
</BrowserRouter>
```

外側から順に意味がある。

1. `BrowserRouter`がURL管理機能を提供する
2. `AuthProvider`が認証状態を提供する
3. `AppRouter`が現在のURLに合う画面を表示する

`AppRouter`の内側にある全画面は、`useAuth()`やReact Routerの機能を使用できる。

## 1.6 Tailwind CSS v4

`vite.config.ts`でTailwindプラグインを読み込む。

```
export default defineConfig({  
  plugins: [react(), tailwindcss()],  
})
```

`index.css`では次の1行でTailwindを導入している。

```
@import "tailwindcss";
```

ComponentではCSSファイルへクラスを定義する代わりに、次のように指定する。

```
className="rounded-xl bg-indigo-600 px-4 py-3 font-semibold text-white"
```

読み方:

- `rounded-xl`: 角を丸くする
- `bg-indigo-600`: 背景を藍色にする
- `px-4`: 左右の余白
- `py-3`: 上下の余白
- `font-semibold`: 少し太い文字
- `text-white`: 白文字

レスポンス指定も`className`内で行う。

```
className="px-4 sm:px-6 lg:px-8"
```

画面幅が広がるにつれて左右余白が増える。

### 現在未使用のApp.css

`src/App.css`にはVite初期テンプレート由来のスタイルが残っているが、現在の`App.tsx`からimportされていない。そのため画面には適用されない。

同様に`react.svg`、`vite.svg`、一部の画像・旧コンポーネントは現在のMVPルートでは使われていない。存在しているコードと、実際に画面から到達できるコードは分けて理解する必要がある。

## 第2章 React基礎

### 2.1 Componentとは

Componentは、画面の一部分を表す関数である。

```
export function LoadingScreen() {
  return (
    <div>
      <p>読み込み中...</p>
    </div>
  )
}
```

React Componentには次の特徴がある。

- 関数として定義する
- JSXを返す
- 他のComponentを組み合わせられる
- Propsを受け取れる
- Stateを持てる

JOBHUNTMANAGERではすべて関数Componentを使用している。

#### Componentを分割する理由

例えばタスク機能は3つへ分かれている。

- **TaskSection**: API通信とState管理
- **TaskForm**: 入力フォーム
- **TaskList**: 一覧表示

1ファイルに全部書くよりも、各Componentの役割が分かりやすく、修正範囲も限定できる。

### 2.2 Props

Propsは、親Componentから子Componentへ渡す値である。

```
type ContentLoadingProps = {
  message?: string
}

export function ContentLoading({
  message = '読み込み中...',
}: ContentLoadingProps) {
  return <p>{message}</p>
}
```

利用側:

```
<ContentLoading message="タスクを読み込み中..." />
```

**message?: string**の?は省略可能という意味である。省略された場合はデフォルト値が使われる。

#### 関数もPropsとして渡せる

```
<TaskForm
  onSubmit={handleCreate}
  onCancel={() => setIsCreating(false)}
/>
```

子Componentは「何を保存するか」を知らなくてもよい。入力が完了したら**onSubmit**を呼ぶだけである。実際にAPIを呼ぶ責務は親の**TaskSection**が持つ。

これを「親が処理を注入する」と考えると理解しやすい。

```
TaskSection
├─ データと処理を持つ
└─ TaskFormへ onSubmit を渡す
    ↓
  Form送信時に呼び返す
```

## 2.3 State

Stateは、Componentが記憶する値である。

```
const [email, setEmail] = useState('')
```

- **email**: 現在の値
- **setEmail**: 値を変更する関数
- **''**: 初期値

入力欄は次のようにStateと接続する。

```
<input
  value={email}
  onChange={(event) => setEmail(event.target.value)}
/>
```

この方式をcontrolled componentという。

```

ユーザーが入力
↓
onChange
↓
setEmail
↓
State変更
↓
Reactが再描画
↓
inputのvalueへ反映

```

### State更新は直接変更しない

配列へデータを追加するとき、次のように新しい配列を作る。

```
setNotes((current) => sortNotes([...current, createdNote]))
```

**current.push(createdNote)**のように元配列を直接変更しない。Reactは参照が変わったことを手掛かりに再描画するため、新しい配列やオブジェクトを作る。

Setも同様である。

```

setUpdatingApplicationIds((current) => {
  const nextIds = new Set(current)
  nextIds.add(card.id)
  return nextIds
})

```

## 2.4 useEffect

**useEffect**は、画面描画そのものではない副作用を実行する。

このアプリでの代表例はAPI取得である。

```

useEffect(() => {
  const fetchKanban = async () => {
    const loadedData = await kanbanApi.get()
    setData(loadedData)
  }

  void fetchKanban()
}, [reloadKey])

```

依存配列**[reloadKey]**の値が変わると再実行される。

再読み込みボタンでは次の処理を行う。

```
setReloadKey((current) => current + 1)
```

数値が変わることでEffectが再実行され、APIを再取得する。これは「再取得用トリガー」としてStateを使うパターンである。

### cleanupとactive

このアプリでは次のパターンを使用する。

```

useEffect(() => {
  let active = true

  const fetchNotes = async () => {
    const loadedNotes = await notesApi.list(applicationId)
    if (active) {
      setNotes(loadedNotes)
    }
  }

  void fetchNotes()

  return () => {
    active = false
  }
}, [applicationId])

```

API通信中に別画面へ移動してComponentが消えた場合、古い通信結果でStateを更新しないようにする。

## 2.5 useMemo

`useMemo`は、計算結果を再利用するHookである。

カンバンでは全応募件数を計算している。

```
const totalApplications = useMemo(
  () =>
    kanbanStatuses.reduce(
      (total, status) => total + data[status].length,
      0,
    ),
  [data],
)
```

`data`が変わったときだけ合計を再計算する。

この計算自体は軽いため、必ず`useMemo`が必要というわけではない。しかし「この値は`data`から導出される値である」ことが明確になる。

面接で聞かれた場合は、次のように説明できる。

全列の応募件数はStateとして別に持たず、カンバンデータから計算しています。データとの不整合を防ぎ、依存する`data`が変わったときだけ計算するため`useMemo`を使っています。

## 2.6 useCallback

`useCallback`は関数の参照を保持するHookである。

応募詳細画面では、子Sectionから件数を受け取る関数へ使っている。

```
const handleTaskCountChange = useCallback((count: number) => {
  setTaskCount(count)
}, [])
```

これを毎回新しい関数として作ると、子ComponentのEffect依存配列が毎回変化してしまう可能性がある。参照を安定させることで、不要なEffect実行を避ける。

## 2.7 useRef

`useRef`は、再描画を起こさずに値やDOM参照を保持する。

用途1: DOMの参照

```
const boardScrollRef = useRef<HTMLDivElement>(null)
```

カンバン上部と本体のスクロール位置を同期するために使用する。

用途2: 最新の通信中IDを同期的に確認

```
const updatingTaskIdsRef = useRef<Set<number>>(new Set())
```

State更新は即時反映ではない場合があるため、同じタスクへの二重操作防止には`ref`も併用している。

## 2.8 Context

Contextは、遠い親子関係でも共通値を共有する仕組みである。

認証情報は多くの画面で必要になる。

```
AuthProvider
├── AppLayout
│   ├── KanbanPage
│   └── ApplicationDetailPage
├── ProtectedRoute
└── GuestRoute
```

毎回Propsで`user`や`logout`を渡すと複雑になるため、Contextを使用する。



## 第3章 TypeScript

### 3.1 TypeScriptを使う理由

JavaScriptだけでは、APIデータの形をコードから判断しにくい。

```
export type User = {
  id: number
  name: string
  email: string
}
```

この型があることで、次の間違いを実行前に検出できる。

```
user.email // emailのタイプミス
user.id.toUpperCase() // numberに文字列用メソッドを使用
```

### 3.2 typeとinterface

どちらもオブジェクトの形を表せる。

```
type User = {
  id: number
  name: string
}
```

```
interface User {
  id: number
  name: string
}
```

現在のJOBHUNTMANAGERでは、ほぼ一貫して**type**を使用している。理由は、Union型や**Omit**などと組み合わせやすく、記述スタイルを統一できるためである。

**interface**が悪いわけではない。チーム内でルールを統一することが重要である。

### 3.3 Union型

複数の決められた値だけを許可する型である。

```
export type AuthStatus =
  | 'loading'
  | 'authenticated'
  | 'unauthenticated'
  | 'error'
```

**status**へ '**logged-in**' などの未定義値は設定できない。

認証画面ではUnion型を条件分岐に利用する。

```
if (status === 'loading') {
  return <LoadingScreen />
}
```

### 3.4 as constとkeyof typeof

応募ステータスでは、表示名オブジェクトから型を作っている。

```
export const applicationStatusLabels = {
  applied: '応募済み',
  document_screening: '書類選考中',
  interview_scheduled: '面接予定',
  offered: '内定',
  rejected: '見送り',
} as const

export type ApplicationStatus =
  keyof typeof applicationStatusLabels
```

**ApplicationStatus**は次のUnion型になる。

```
'applied'
| 'document_screening'
| 'interview_scheduled'
| 'offered'
| 'rejected'
```

型と表示名を別々に重複定義しないため、追加・変更時の漏れを減らせる。

## 3.5 このアプリで使う便利な型

### Record

```
export type KanbanData =
  Record<ApplicationStatus, KanbanCardData[]>
```

すべての応募ステータスをキーに持ち、それぞれカード配列を保存する。

```
{
  applied: [],
  document_screening: [],
  interview_scheduled: [],
  offered: [],
  rejected: [],
}
```

### Omit

```
export type ApplicationDetail =
  Omit<ApplicationSummary, 'job_posting'> & {
    job_posting: DetailedJobPosting
  }
```

既存型から特定項目だけ除き、より詳細な型へ差し替える。

### Partial

```
export type UpdateTaskInput = Partial<TaskInput> & {
  completed?: boolean
}
```

更新時は全項目を必須にせず、一部だけ送信できる。完了切り替えでは次だけ送る。

```
{ completed: true }
```

### Generics

```
export type ApiResponse<T> = {
  data: T
}
```

Tは利用時に決まる型である。

```
ApiResponse<User>
ApiResponse<Task[]>
ApiResponse<KanbanData>
```

同じ `{ data: ... }` 形式を再利用できる。

### null

API上で未設定になり得る項目は明示的に `null` を含める。

```
due_at: string | null
meeting_url: string | null
```

表示時にはチェックが必要である。

```
task.due_at ? formatDueAt(task.due_at) : '未設定'
```

## 3.6 API型とフォーム型を分ける理由

Taskを例にすると、APIデータはsnake caseで日時はISO 8601である。

```
type Task = {
  due_at: string | null
}
```

フォームではHTML inputに合わせてcamelCaseと `datetime-local` 形式を使う。

```
type TaskFormValues = {
  dueAt: string
}
```

変換関数が両者の橋渡しをする。

```
export const toTaskInput = (values: TaskFormValues): TaskInput => ({
  title: values.title.trim(),
  due_at: values.dueAt
    ? new Date(values.dueAt).toISOString()
    : '',
  priority: values.priority,
})
```

UI都合の型とAPI都合の型を混ぜないことが、保守性につながる。

## 第4章 React Router

### 4.1 Routerの全体像

App.tsxでBrowserRouterを提供し、router.tsxでルート进行定義する。

#### 図解

1. BrowserRouter → ProtectedRoute
2. GuestRoute → AuthLayout
3. AuthLayout → /login
4. AuthLayout → /signup
5. ProtectedRoute → AppLayout
6. AppLayout → /kanban
7. AppLayout → /applications/:id

### 4.2 router.tsx

```
<Route element={}<ProtectedRoute />>
  <Route element={}<AppLayout />>
    <Route path="/kanban" element={}<KanbanPage /> />
    <Route
      path="/applications/:id"
      element={}<ApplicationDetailPage />
    />
  </Route>
</Route>
```

この構造は、認証済み画面へ共通の条件とレイアウトを適用している。

/applications/:idのidはURLパラメータである。

```
const { id } = useParams()
```

URLが/applications/20なら、idは文字列"20"になる。

### 4.3 ネストルーティング

親Routeの中に子Routeを置くことをネストルーティングという。

認証済みルートでは次の順で表示される。

```
ProtectedRoute
├── AppLayout
│   └── KanbanPage または ApplicationDetailPage
```

### 4.4 Outlet

Outletは、現在一致した子Routeの表示位置である。

AppLayout.tsx:

```
<header>...</header>
<main>
  <Outlet />
</main>
```

ヘッダーは固定して、中身だけをカンバンや応募詳細へ入れ替えられる。

AuthLayoutも同じ仕組みで、ログインと新規登録に共通の背景やカードを提供する。

## 4.5 ProtectedRoute

```
if (status === 'loading') {
  return <LoadingScreen />
}

if (status === 'error') {
  return <AuthErrorScreen />
}

if (status === 'unauthenticated') {
  return <Navigate to="/login" replace state={{ from: location }} />
}

return <Outlet />
```

認証状態ごとに表示を切り替える。

| status          | 表示                              |
|-----------------|---------------------------------|
| loading         | 認証確認中                           |
| error           | 通信エラー・再試行                       |
| unauthenticated | <a href="/login">/login</a> へ移動 |
| authenticated   | 子画面を表示                          |

`replace`を付けると、履歴を置き換える。戻るボタンで保護画面とログイン画面を行き来しにくくなる。

`state={{ from: location }}`は、どこからログインへ移動したかを保持するための情報である。現在のログイン後処理では常にカンバンへ進むため、この情報は将来「元の画面へ戻る」機能に利用できる。

## 4.6 GuestRoute

未認証ユーザー専用画面を守る。

```
if (status === 'authenticated') {
  return <Navigate to="/kanban" replace />
}
```

ログイン済みユーザーが</login>を開いても、カンバンへ戻る。

## 4.7 画面遷移

通常リンク:

```
<Link to={`/${applications}/${card.id}`}>
```

現在のURLに応じたスタイル:

```
<NavLink
  to="/kanban"
  className={({ isActive }) =>
    isActive ? 'bg-indigo-50' : 'text-slate-600'
  }
/>
```

処理完了後の移動:

```
const navigate = useNavigate()
navigate('/kanban', { replace: true })
```

応募削除後は、存在しなくなった詳細画面を履歴に残さずカンバンへ戻る。

## 第5章 認証機能

### 5.1 認証の登場人物

| ファイル               | 役割                    |
|--------------------|-----------------------|
| AuthProvider.tsx   | 認証状態と操作を全画面へ提供        |
| AuthContext.ts     | Contextの型と本体          |
| useAuth.ts         | Contextを安全に取り出す       |
| authApi.ts         | Rails認証APIを呼び         |
| tokenStorage.ts    | JWTをsessionStorageへ保存 |
| client.ts          | 全APIへJWTを付与           |
| ProtectedRoute.tsx | 未認証時にログインへ移動          |
| GuestRoute.tsx     | 認証済みならカンパへ移動          |

### 5.2 JWTログインの流れ

#### 図解

1. ユーザー → LoginPage メールとパスワードを送信
2. LoginPage → AuthProvider login(input)
3. AuthProvider → authApi authApi.login(input)
4. authApi → apiClient POST /api/v1/auth/sign\_in
5. apiClient → Rails API JSON送信
6. Rails API → apiClient User JSON + Authorization header
7. apiClient → authApi AxiosResponse
8. authApi → AuthProvider user + token
9. AuthProvider → AuthProvider sessionStorageへtoken保存
10. AuthProvider → AuthProvider user/statusを更新
11. AuthProvider → LoginPage 再描画

### 5.3 AuthContext

```
export type AuthContextValue = {
  user: User | null
  status: AuthStatus
  login: (input: LoginInput) => Promise<void>
  signup: (input: SignupInput) => Promise<void>
  logout: () => Promise<void>
  retryAuthentication: () => Promise<void>
}
```

認証に必要な値と操作を1か所へまとめている。

```
export const AuthContext =
  createContext<AuthContextValue | undefined>(undefined)
```

初期値を`undefined`にすることで、Provider外で誤使用した場合に検出できる。

### 5.4 useAuth

```
export const useAuth = () => {
  const context = useContext(AuthContext)

  if (!context) {
    throw new Error(
      'useAuthはAuthProvider内で使用してください。',
    )
  }

  return context
}
```

各Componentが毎回`useContext(AuthContext)`と安全確認を書く必要がない。

利用側:

```
const { user, logout } = useAuth()
```

## 5.5 AuthProviderのState

```
const [user, setUser] = useState<User | null>(null)
const [status, setStatus] = useState<AuthStatus>('loading')
```

`user`だけで認証状態を判断しない点が重要である。

`user === null`には複数の意味があり得る。

- 未認証
- 認証確認中
- 通信エラー

そのため`status`を別に持つ。

## 5.6 sessionStorage

```
const TOKEN_KEY = 'jobhuntmanager.jwt'

export const tokenStorage = {
  get(): string | null {
    return sessionStorage.getItem(TOKEN_KEY)
  },
  set(token: string): void {
    sessionStorage.setItem(TOKEN_KEY, token)
  },
  remove(): void {
    sessionStorage.removeItem(TOKEN_KEY)
  },
}
```

sessionStorageの特徴:

- タブ単位で保存される
- タブを閉じると削除される
- JavaScriptから読み書きできる
- Cookieのように自動送信されない

このアプリで選んだ理由:

- MVPのSPAで扱いやすい
- localStorageより保存期間が短い
- タブを閉じた後まで認証情報を残さない

注意点:

- XSSがあるとJavaScriptからJWTを盗まれる
- Reactの自動エスケープを活用する
- `dangerouslySetInnerHTML`を使わない
- `VITE` 環境変数へ秘密情報を入れない

## 5.7 AuthorizationヘッダーからJWTを取得

Railsはログイン成功時、レスポンスヘッダーへJWTを返す。

```
const authorization = response.headers.authorization
const token = authorization
  .replace(/Bearer\s+/i, '')
  .trim()
```

JSON本文ではなくヘッダーから取得する点が、バックエンド仕様と一致している。

## 5.8 起動時の認証復元

画面再読み込みではReactのStateが消える。しかしsessionStorageのJWTは残る。

```
const restoreSession = useCallback(async () => {
  if (!tokenStorage.get()) {
    setUser(null)
    setStatus('unauthenticated')
    return
  }

  setStatus('loading')

  try {
    const currentUser = await authApi.me()
    setUser(currentUser)
    setStatus('authenticated')
  } catch (error) {
    if (isUnauthorizedError(error)) {
      clearSession()
      return
    }

    setUser(null)
    setStatus('error')
  }
}, [clearSession])
```

重要な分岐:

### JWTがない

未認証として扱う。

### /auth/meが成功

ユーザー情報を復元する。

### 401

JWTが無効・期限切れなので削除する。

### 通信障害・500

JWTは削除しない。サーバーが一時的に落ちているだけかもしれないため、**error**状態にして再試行画面を出す。

これは認証情報を不用意に消さないための設計である。

## 5.9 StrictModeと初期化

```
const initialized = useRef(false)

useEffect(() => {
  if (initialized.current) {
    return
  }

  initialized.current = true
  void restoreSession()
}, [restoreSession])
```

開発中のStrictModeで初期化処理が重複しないようにしている。

## 5.10 ログアウト

```
const logout = async () => {
  try {
    await authApi.logout()
    clearSession()
  } catch (error) {
    if (isUnauthorizedError(error)) {
      clearSession()
      return
    }

    throw error
  }
}
```

| 結果       | 処理               |
|----------|------------------|
| 成功       | JWT削除、未認証へ       |
| 401      | すでに無効なのでJWT削除    |
| 通信障害・500 | JWTを残し、画面へエラーを返す |

サーバーへ到達していないのにローカルだけログアウトすると、利用者が再試行できなくなる可能性がある。そのため失敗種別を分けている。

## 第6章 AxiosとAPI通信

### 6.1 なぜAPI処理を分けるのか

Component内に直接URLを書くと、UIと通信が混ざる。

```
// 避けたいイメージ
await axios.post('/api/v1/tasks/...')
```

このアプリでは次のように呼ぶ。

```
const createdTask =
  await tasksApi.create(applicationId, input)
```

Componentは「タスクを作る」とだけ理解し、URLやJSON構造は`tasksApi.ts`へ任せる。

### 6.2 共通Axios client

```
export const apiClient = axios.create({
  baseURL: apiBaseUrl,
  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json',
  },
})
```

VITE API BASE URLは`http://localhost:3000`であり、各APIモジュールが`/api/v1/...`を付ける。

```
baseURL
http://localhost:3000

request path
/api/v1/kanban

実際のURL
http://localhost:3000/api/v1/kanban
```

### 6.3 Request Interceptor

Interceptorは、リクエストやレスポンスへ共通処理を差し込む仕組みである。

```
apiClient.interceptors.request.use((config) => {
  const token = tokenStorage.get()

  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }

  return config
})
```

各APIで毎回Authorizationを書く必要がない。

```
tasksApi.update()
↓
Request Interceptor
↓ JWTを付与
Authorization: Bearer ...
↓
Rails API
```

### 6.4 Response Interceptor

```
apiClient.interceptors.response.use(
  (response) => response,
  (error: unknown) => {
    if (
      axios.isAxiosError(error) &&
      error.response?.status === 401
    ) {
      tokenStorage.remove()
      unauthorizedHandler?.()
    }

    return Promise.reject(error)
  },
)
```

どのAPIで401が返っても、認証状態を解除できる。

`unauthorizedHandler`にはAuthProviderの`clearSession`が登録される。

```
useEffect(() => {
  setUnauthorizedHandler(clearSession)
  return () => setUnauthorizedHandler(null)
}, [clearSession])
```

Axios層がReact Stateを直接操作せず、関数を登録する形で疎結合にしている。



## 6.5 APIモジュール

タスク一覧:

```
async list(
  applicationId: number | string,
): Promise<Task[]> {
  const response =
    await apiClient.get<ApiResponse<Task[]>>(`
      /api/v1/applications/${applicationId}/tasks`,
    )

  return response.data.data
}
```

読み方:

1. `number | string`なのでURL由来の文字列IDにも数値IDにも対応
2. Axiosレスポンス型を指定
3. 共通形式`{ data: ... }`から中身だけ返す
4. UIはAxiosResponseを意識しなくてよい

## 6.6 APIごとのJSONルートキー

RailsのStrong Parametersに合わせている。

```
{ task: input }
{ interview: input }
{ note: input }
{ application: input }
```

例えばタスク作成:

```
{
  "task": {
    "title": "面接資料を準備",
    "priority": "high"
  }
}
```

## 6.7 エラー変換

`getApiErrorMessage`はunknown型のエラーを安全に文字列へ変換する。

```
if (!error.response) {
  return 'サーバーに接続できません。APIが起動しているか確認してください。'
}

return error.response.data?.error?.message
  ?? DEFAULT_ERROR_MESSAGE
```

`getApiValidationErrors`は項目別エラーを返す。

```
Record<string, string[]>
```

例:

```
{
  "title": ["を入力してください"],
  "due_at": ["は正しい日時を指定してください"]
}
```

## 6.8 VitestによるAPIテスト

実際のRailsへ通信せず、`apiClient`をmockする。

```
vi.mock('../shared/api/client', () => ({
  apiClient: apiClientMock,
})))
```

テストではURLとJSONを確認する。

```
expect(apiClientMock.patch).toHaveBeenCalledWith(
  '/api/v1/tasks/40',
  { task: { completed: true } },
)
```

これにより、フロントとRailsの契約を誤って変更したときに検出しやすい。

## 第7章 カンバン画面

### 7.1 Component構成

```
KanbanPage
├─ KanbanBoard
│  └─ KanbanColumn × 5
│     └─ KanbanQuickAddForm (応募済み列のみ)
│     └─ KanbanCard × 件数
```

役割:

| Component          | 役割                    |
|--------------------|-----------------------|
| KanbanPage         | API取得、全体State、ステータス更新 |
| KanbanBoard        | 5列表示、横スクロール同期         |
| KanbanColumn       | 列タイトル、件数、カード一覧        |
| KanbanCard         | 会社名、応募日、ステータス選択       |
| KanbanQuickAddForm | 会社名と応募日による簡易登録        |

### 7.2 カンバンの型

```
export type KanbanData =
  Record<ApplicationStatus, KanbanCardData[]>
```

5列を必ず持つ。

```
export const emptyKanbanData = (): KanbanData => ({
  applied: [],
  document_screening: [],
  interview_scheduled: [],
  offered: [],
  rejected: [],
})
```

関数として新しいオブジェクトを返すため、複数Stateで同じオブジェクト参照を共有しない。

### 7.3 データ取得

```
const [data, setData] =
  useState<KanbanData>(emptyKanbanData)
```

初期値として関数を渡している。Reactは初回だけ実行する。

取得:

```
const loadedData = await kanbanApi.get()
setData(loadedData)
```

API側でも空列を返す仕様だが、フロントでも不足列を空配列で補う。

```
kanbanStatuses.forEach((status) => {
  data[status] = response.data.data[status] ?? []
})
```

バックエンドレスポンスが一部欠けても、画面が壊れにくい。

## 7.4 簡易応募登録

応募済み列の+ボタンでフォームを開く。

```
{status === 'applied' && (
  <button onClick={onOpenQuickAdd}>+</button>
)}
```

入力値:

- 会社名
- 応募日

送信:

```
const card = await kanbanApi.createApplication({
  company_name: trimmedCompanyName,
  applied_on: appliedOn,
})

onCreated(card)
```

親のKanbanPageは返されたカードを応募済み列へ追加する。

```
setData((current) => ({
  ...current,
  applied: sortCards([card, ...current.applied]),
}))
```

一覧全体を再取得せず、APIが返した1件をStateへ追加するため、操作後すぐに画面へ反映できる。

Rails内部ではCompany、JobPosting、Applicationが作られるが、フロントは簡易登録APIを1回呼び出すだけである。この責務分担により、途中の作成失敗やトランザクションをRailsへ任せられる。

## 7.5 ステータス変更

```
onChange={(event) =>
  void onStatusChange(
    card,
    event.target.value as ApplicationStatus,
  )
}
```

処理の流れ:

### 図解

1. User → KanbanCard selectを変更
2. KanbanCard → KanbanPage onStatusChange(card, status)
3. KanbanPage → KanbanPage IDを通信中Setへ追加
4. KanbanPage → kanbanApi updateStatus
5. kanbanApi → Rails PATCH status
6. Rails → kanbanApi 更新済みカード
7. kanbanApi → KanbanPage updatedCard
8. KanbanPage → KanbanPage 全列から旧カードを除去
9. KanbanPage → KanbanPage 新しい列へカードを追加
10. KanbanPage → KanbanPage 通信中SetからIDを削除

### 同じカードへの二重操作防止

```
if (
  nextStatus === card.status ||
  updatingApplicationIds.has(card.id)
) {
  return
}
```

通信中はselectも無効化する。

### 失敗時

成功するまで元のStateを変更していないため、失敗時は元の列・値がそのまま残る。カードへエラーメッセージだけ表示する。

```
setCardErrors((current) => ({
  ...current,
  [card.id]: `${getApiErrorMessage(error)} 元のステータスに戻しました。`,
}))
```

## 7.6 列内の並び順

```
const sortCards = (cards: KanbanCardData[]) =>
  [...cards].sort((left, right) => {
    const difference =
      new Date(right.updated_at).getTime() -
      new Date(left.updated_at).getTime()

    return difference !== 0
      ? difference
      : right.id - left.id
  })
```

要件どおり:

1. `updated_at` DESC
2. 同時刻なら `id` DESC

`[...cards]`でコピーしてからsortする。`sort()`は元配列を変更するため、State配列を直接変更しない工夫である。

## 7.7 横スクロール

5列を表示するため、狭い画面では横スクロールが必要になる。

`KanbanBoard`では上部スクロールバーと本体を同期している。

```
const topScrollRef = useRef<HTMLDivElement>(null)
const boardScrollRef = useRef<HTMLDivElement>(null)

target.scrollLeft =
  event.currentTarget.scrollLeft
```

`ResizeObserver`で幅の変化を監視し、横にはみ出す場合だけ上部スクロールを表示する。

```
setIsOverflowing(
  content.scrollWidth > board.clientWidth,
)
```

1920px級の画面では`2xl:w-full`と`2xl:flex-1`により5列が横幅へ収まり、不要なスクロールバーを表示しない。

## 7.8 再描画を説明する

面接で「selectを変更したら、なぜカードが別列へ移るのか」と聞かれた場合:

API成功後に`KanbanData`の新しいオブジェクトを作り、旧カードを全列から除去して更新後ステータスの配列へ追加しています。`setData`でStateの参照が変わるためReactが再描画し、`KanbanColumn`が新しい`data[status]`を受け取ってカードの表示列が変わります。

## 第8章 応募詳細画面

### 8.1 画面全体

`ApplicationDetailPage`は次を担当する。

- URLから応募IDを取得
- 応募詳細を取得
- 応募日を更新
- 応募を削除
- 面接・タスク・メモSectionを配置
- 各Sectionの件数を表示

```
ApplicationDetailPage
├─ 応募基本情報
├─ 企業情報
├─ 関連件数
├─ InterviewSection
├─ TaskSection
└─ NoteSection
```

### 8.2 詳細取得

```
const { id } = useParams()
const [application, setApplication] =
  useState<ApplicationDetail | null>(null)

const loadedApplication =
  await applicationsApi.get(id)

setApplication(loadedApplication)
setAppliedOn(loadedApplication.applied_on)
setInterviewCount(
  loadedApplication.interviews.length,
)
```

親APIの詳細レスポンスから初期件数を表示し、その後各Sectionが専用一覧APIを取得して最新件数を通知する。

### 8.3 応募日更新

```
const updatedApplication =
  await applicationsApi.update(id, {
    applied_on: appliedOn,
  })
```

通常の応募更新APIでは`applied_on`だけを送る。ステータスはカンバン専用APIで変更する設計である。

成功後は必要な項目だけを置き換える。

```
setApplication((current) =>
  current
  ? {
    ...current,
    applied_on: updatedApplication.applied_on,
    updated_at: updatedApplication.updated_at,
  }
  : current,
)
```

### 8.4 応募削除

確認表示を出し、ユーザーが確定してからDELETEする。

```
await applicationsApi.destroy(id)
navigate('/kanban', { replace: true })
```

Rails側では面接・タスク・メモも連鎖削除される。フロントはその影響を説明文として表示している。

## 8.5 Section / Form / Listパターン

面接・タスク・メモは共通パターンを使う。

### 図解

1. Section: API・State → List: 一覧・操作ボタン
2. Form: 入力・検証表示 -- onSubmit(input) --> Section: API・State
3. List: 一覧・操作ボタン -- onEdit / onDelete --> Section: API・State
4. Section: API・State → API module

### Section

- ・ 一覧取得
- ・ 作成・更新・削除
- ・ ローディング
- ・ 通信エラー
- ・ 編集対象
- ・ 削除確認対象
- ・ 通信中ID

### Form

- ・ 入力値
- ・ 送信中状態
- ・ APIバリデーションエラー
- ・ API用データへの変換

### List

- ・ 空状態
- ・ データ表示
- ・ 編集ボタン
- ・ 削除確認
- ・ 通信中の操作無効化

このパターンを覚えると、新しい「連絡履歴」機能なども同じ構造で追加できる。

## 8.6 TaskSection

### 並び順

1. 未完了
2. 完了
3. 期限ありは期限の早い順
4. 期限なしは後
5. 同条件ならID順

### 完了切り替え

```
const updatedTask = await tasksApi.update(
  task.id,
  {
    completed: task.completed_at === null,
  },
)
```

DB上の`completed_at`を直接送らず、API用のboolean`completed`を送る。

### overdue

期限超過判定はRailsが行い、フロントは`task.overdue`を表示する。

```
{task.overdue && <span>期限超過</span>}
```

時刻判定を複数クライアントへ分散せず、サーバー側を正としている。

## 8.7 InterviewSection

面接では型の制限が特に重要である。

```
type InterviewType =
  | 'casual'
  | 'first'
  | 'second'
  | 'final'
  | 'other'
```

selectの選択肢とAPI enumを一致させる。

### 日時変換

HTMLの`datetime-local`はタイムゾーン情報を持たない。

送信時:

```
new Date(values.scheduledAt).toISOString()
```

UTCのISO 8601へ変換する。

編集時:

```
const localDate = new Date(
  date.getTime() -
  date.getTimezoneOffset() * 60_000,
)
return localDate.toISOString().slice(0, 16)
```

APIの日時をローカル入力欄の形式へ変換する。

### Application statusは自動変更しない

面接の作成・更新は`interviewsApi`だけと呼ぶ。応募ステータス更新APIは呼んでいない。

これにより「面接を登録したら勝手にカンバンが移動する」という副作用を避けている。

## 8.8 NoteSection

API名は`content`、DBカラムは`body`だが、フロントはDBを意識しない。

```
type Note = {
  content: string
}
```

フォームでは最大10,000文字を制限する。

```
<textarea
  required
  maxLength={10_000}
  value={values.content}
/>
```

文字数:

```
{values.content.length.toLocaleString()}
/ 10,000文字
```

ブラウザ側の制限はUI向上のためであり、Rails側にも同じバリデーションが必要である。

## 8.9 親への件数通知

```
useEffect(() => {
  if (!isLoading && !errorMessage) {
    onNoteCountChange?.(notes.length)
  }
}, [
  errorMessage,
  isLoading,
  notes.length,
  onNoteCountChange,
])
```

State更新関数の内部で親Callbackを呼ばず、Effectから通知している。

理由:

- State updaterを純粋関数に保つ
- StrictModeで副作用が重複する危険を減らす
- 「State変更」と「親への通知」を分ける

## 8.10 複数通信状態の管理

単一の`isUpdating`だけでは、複数項目を同時操作したときに区別できない。

```
const [updatingTaskIds, setUpdatingTaskIds] =
  useState<Set<number>>>(() => new Set())
```

例えばタスク1とタスク2を同時更新する場合:

```
Set { 1, 2 }
```

タスク1が終了:

```
Set { 2 }
```

タスク2は引き続き操作不可になる。

### なぜrefも使うのか

```
const updatingTaskIdsRef =
  useRef<Set<number>>>(new Set())
```

Stateは再描画用、refは同一イベント期間中にも最新値を同期的に確認するために使う。

```
if (updatingTaskIdsRef.current.has(taskId)) {
  return false
}
```

二重クリックなどで同じIDの通信が重なるのを防ぐ。

## 8.11 現在MVP画面で使われていないコード

以下はコードとして存在するが、現在の`router.tsx`から到達できない。

- `ApplicationCreatePanel`
- `CompanyCreatePanel`
- `JobPostingForm`
- `JobPostingCard`
- `companiesApi`
- `jobPostingsApi`
- 求人管理用の型

以前の求人管理UIや、既存APIを利用するための実装である。現在のMVPはカンバン簡易登録を中心にしたため、画面ルートは削除されている。

面接では次のように説明できる。

求人APIと関連コンポーネントはバックエンド互換性のため残っていますが、現在のMVP導線では会社名と応募日の簡易登録を優先し、ルーティングからは外しています。今後求人管理画面を再導入するときに再利用可能です。

ただし、長期的には未使用コードを残すコストもあるため、再利用予定がない場合は削除を検討する。



## 第9章 エラーハンドリング

### 9.1 エラーの種類

| 状況           | HTTP       | フロントの扱い       |
|--------------|------------|---------------|
| JWTなし・無効     | 401        | 認証解除、ログインへ    |
| 対象なし・他ユーザー所有 | 404        | 共通メッセージ表示     |
| 入力エラー        | 422        | 全体メッセージ+項目別表示 |
| レート制限        | 429        | APIメッセージ表示    |
| サーバーエラー      | 500        | 共通エラー表示       |
| 通信できない       | responseなし | API起動・通信環境の案内 |

### 9.2 422 バリデーションエラー

Railsレスポンス:

```
{
  "error": {
    "code": "validation_error",
    "message": "入力内容を確認してください",
    "details": {
      "content": ["を入力してください"]
    }
  }
}
```

フォーム:

```
setErrorMessage(getApiErrorMessage(error))
setFieldErrors(getApiValidationErrors(error))
```

項目別:

```
const contentError =
  (fieldErrors.content ?? []).join('、')
```

全体エラーだけでなく、どの項目を直すべきか分かる。

### 9.3 401

Axios Interceptorが共通処理する。

```
任意のAPIで401
↓
sessionStorageからJWT削除
↓
AuthProviderのclearSession
↓
status = unauthenticated
↓
ProtectedRoute
↓
/login
```

各Pageで401処理を重複実装しない。

### 9.4 404

Railsは他ユーザー所有のデータも404にする。フロントは「存在しない」のか「権限がない」のかを区別しない。

これにより、IDを試して他ユーザーのデータ存在を推測しにくくする。

### 9.5 500と通信障害

`getApiErrorMessage`は、レスポンス自体がない場合を区別する。

```
if (!error.response) {
  return 'サーバーに接続できません。APIが起動しているか確認してください。'
}
```

一覧画面では再読み込みボタンを出す。

```
<InlineAlert message={errorMessage} />
<button onClick={handleReload}>
  再読み込み
</button>
```

### 9.6 429

Rack::Attackの共通JSONメッセージは`getApiErrorMessage`で表示できる。

現在のフロントは`Retry-After`を読み取ってカウントダウン表示はしていない。MVPではメッセージ表示で対応し、将来的に再試行可能時刻を表示できる。

### 9.7 ローディング状態

一覧読み込み:

```
{isLoading && (
  <ContentLoading message="メモを読み込み中..." />
)}
```

ボタン送信:

```
disabled={isSubmitting}
{isSubmitting ? '保存中...' : submitLabel}
```

通信中であることを見せ、二重送信を防止する。

### 9.8 finally

```
try {
  await onSubmit(input)
} catch (error) {
  setErrorMessage(...)
} finally {
  setIsSubmitting(false)
}
```

成功・失敗のどちらでも送信中状態を解除する。`finally`がないと、エラー時にボタンが永久に無効になる可能性がある。

## 第10章 面接で聞かれそうな質問

### Q1. なぜContextを使いましたか

回答例:

ログインユーザー、認証状態、ログイン・ログアウト処理は、ルートガード、ヘッダー、ログイン画面など複数箇所で必要です。Propsを何階層も渡すと管理が複雑になるため、AuthContextでアプリ全体へ提供しました。業務データまでContextへ入れず、認証という横断的な状態に限定しています。

### Q2. なぜReduxを使わなかったのですか

回答例:

MVPではグローバルに共有する状態が主に認証情報だけで、カンバンや応募詳細のデータは各Page内で完結しています。Reduxを導入するとactionやstoreなどの学習・実装コストが増えるため、認証はContext、画面データはuseStateで十分と判断しました。複雑なキャッシュ共有や画面横断状態が増えた場合はRedux ToolkitやTanStack Queryを検討します。

### Q3. なぜAxiosを使いましたか

回答例:

baseURLや共通ヘッダーを設定し、InterceptorでJWT付与と401処理を集約しやすいためです。AxiosErrorの判定も利用して、通信障害とAPIエラーを分けています。fetchでも実装できますが、このアプリでは共通処理の見通しを優先しました。

### Q4. なぜsessionStorageですか

回答例:

SPAでBearer JWTを扱うために使用しています。localStorageより保持期間が短く、タブを閉じると消える点を選びました。一方でJavaScriptから参照できるためXSSには弱く、dangerouslySetInnerHTMLを使わず、Reactの自動エスケープを利用しています。本番ではCSPも設定します。

### Q5. Cookie認証ではない理由は何ですか

回答例:

現在の設計要件がRails APIとReact SPA間のBearer JWT認証だからです。Authorizationヘッダーで明示的に送信し、Devise JWTのJTMatcherでログアウト失効を行っています。HttpOnly CookieにはXSS耐性の利点がありますが、CSRFやCookie属性を含め別設計になるためMVPでは採用していません。

### Q6. React Routerを選んだ理由は何ですか

回答例:

SPAでURLと画面を対応させ、BrowserRouter、ネストルーティング、Outlet、Navigateを使って認証済み・未認証ルートを整理できるためです。AppLayoutやAuthLayoutも親Routeとして共通化しています。

### Q7. ProtectedRouteはどのように動きますか

回答例:

AuthProviderのstatusを見て、確認中ならLoading、通信障害なら再試行画面、未認証なら/loginへNavigateし、認証済みならOutletで子画面を表示します。userがnullかだけでは確認中と未認証を区別できないため、4状態のUnion型を使っています。

### Q8. /auth/meはなぜ必要ですか

回答例:

ブラウザ再読み込みでReact Stateは消えますが、sessionStorageのJWTは残ります。JWTを付けて/auth/meを呼ぶことで、サーバー側で有効性を確認し、ユーザー情報と認証状態を復元します。

### Q9. 401と500でJWTの扱いを変えている理由は何ですか

回答例:

401はJWTが無効であることを示すため削除します。一方500や通信障害はJWT自体が無効とは限らないため削除せず、再試行可能な認証エラー状態にしています。

### Q10. カンバンのステータス変更はどう再描画されますか

回答例:

PATCH成功後、旧カードをすべての列から除去し、APIが返した新ステータスの列へ追加した新しいKanbanDataをsetDataします。State参照が変わるためReactが再描画し、カードの列が移動します。

### Q11. なぜ配列を直接sortしないのですか

回答例:

JavaScriptのsortは元配列を変更します。React Stateを直接変更すると更新検知や予測可能性を損なうため、スプレッド構文でコピーしてからsortしています。

### Q12. なぜ通信中IDをSetで管理していますか

回答例:

1つのbooleanでは、複数タスクを同時更新したときにどの項目が通信中か区別できません。SetへIDを保存することで、項目単位でボタンを無効化し、1件の通信完了が別件の通信状態を解除しないようにしています。

## Q13. TypeScriptで工夫した点は何ですか

回答例:

APIのenumをUnion型にし、`keyof typeof`で表示名オブジェクトから型を生成しています。またAPIデータ型とフォーム型を分け、日時やsnake\_caseの変換を関数へ集約しています。共通レスポンスは`ApiResponse<T>`というGenericsで表現しています。

## Q14. フォームのバリデーションはどこで行いますか

回答例:

`required`や`maxLength`でブラウザ側の入力支援を行い、Rails側でも必ず検証します。Railsの422レスポンスに含まれるdetailsをフィールド別エラーとして表示しています。フロントの検証はUX、バックエンドの検証はデータ保護という役割分担です。

## Q15. なぜAPI通信をComponentから分離しましたか

回答例:

Componentは表示とユーザー操作、APIモジュールはURL・HTTPメソッド・JSON構造を担当させるためです。UIから通信仕様を分離することで、テストしやすく、API変更時の修正箇所も限定できます。

## Q16. useEffectのactiveフラグは何のためですか

回答例:

API通信中に画面遷移してComponentが破棄された場合、古い通信結果でStateを更新しないためです。cleanupでactiveをfalseにし、完了時に確認しています。

## Q17. useMemoとuseCallbackは必須ですか

回答例:

すべての計算や関数に必要ではありません。このアプリでは、useMemoをカンバン件数というdataからの導出値に使い、useCallbackを子ComponentのEffect依存になる件数通知関数など、参照の安定が意味を持つ場所に限定しています。

## Q18. Tailwind CSSの利点は何ですか

回答例:

Componentを見ながらスタイルを確認でき、レスポンスやhover、disabled状態を同じclassNameで定義できます。画面間で色・余白・角丸のルールを揃えやすい点も利点です。一方でclassNameが長くなるため、繰り返す入力スタイルは定数へまとめています。

## Q19. 現在の改善点は何ですか

回答例:

コンポーネントテストがAPIモジュールテストに比べて少ないため、React Testing Libraryでフォーム操作やエラー表示を追加したいです。また一覧取得とキャッシュが増えた場合はTanStack Queryを検討します。現在未使用の求人管理コンポーネントも、再利用方針を決めて整理したいです。

## Q20. フロントだけで他ユーザーアクセスを防げますか

回答例:

防げません。フロントの表示制御は回避できるため、Rails側でcurrent\_userを起点に検索し、他ユーザーのIDは404にしています。フロントは認証UXを担当し、最終的な認可はバックエンドで保証します。

## 付録A 全ファイル役割一覧

### A.1 ルート設定

| ファイル               | 役割                                 |
|--------------------|------------------------------------|
| package.json       | 依存パッケージとdev/bui ld/test/ lintコマンド  |
| package-lock.json  | 実際に解決された依存バージョンを固定                 |
| vite.config.ts     | React + Tailwind プライイン設定           |
| tsconfig.json      | app用とnode用TypeScript設定の参照          |
| tsconfig.app.json  | srcの型チェック設定                        |
| tsconfig.node.json | Vite設定ファイル用の型チェック                  |
| eslint.config.js   | TypeScript, Hooks, Vite向け lint     |
| .env.example       | Rails APIの接続先例                     |
| .gitignore         | node_modules, dist, 環境変数などを除外      |
| index.html         | Reactを挿入するroot要素                   |
| frontend/README.md | Viteテンプレート由来。プロジェクト説明はルートREADMEが中心 |

### A.2 起動・全体構成

| ファイル          | 役割                                          |
|---------------|---------------------------------------------|
| src/main.tsx  | Reactをrootへ描画、StrictMode                    |
| src/App.tsx   | BrowserRouter, AuthProvider, AppRouterの組み立て |
| src/index.css | Tailwind導入、フォント、全体CSS、カンパンスタイル              |
| src/App.css   | Vite初期スタイル。現在未import                        |

### A.3 app

| ファイル                           | 役割                       |
|--------------------------------|--------------------------|
| app/router.tsx                 | URLとPage、認証ガード、Layoutを定義 |
| app/layouts/AppLayout.tsx      | 認証後ヘッダー、ナビ、ログアウト         |
| app/layouts/AuthLayout.tsx     | ログイン・登録画面の共通レイアウト        |
| app/providers/AuthContext.ts   | 認証Contextの型              |
| app/providers/AuthProvider.tsx | 認証State、復元、ログイン、登録、ログアウト |

### A.4 pages

| ファイル                      | 役割                |
|---------------------------|-------------------|
| LoginPage.tsx             | ログインフォーム          |
| SignupPage.tsx            | ユーザー登録フォーム        |
| KanbanPage.tsx            | カンパニ全体StateとAPI制御 |
| ApplicationDetailPage.tsx | 応募詳細と関連Sectionの統合 |
| NotFoundPage.tsx          | 一致しないURLの404画面    |

### A.5 auth

| ファイル                                  | 役割                      |
|---------------------------------------|-------------------------|
| features/auth/api/authApi.ts          | 登録・ログイン・ae・ログアウトAPI     |
| features/auth/hooks/useauth.ts        | AuthContext用Custom Hook |
| features/auth/storage/tokenStorage.ts | sessionStorageでのAPI操作   |
| features/auth/types.ts                | User、入力、AuthState型      |

### A.6 kanban

| ファイル                             | 役割                  |
|----------------------------------|---------------------|
| features/kanban/types.ts         | 5ステータス、カード、レスポンス型   |
| features/kanban/api/kanbanApi.ts | 取得、簡易登録、status更新    |
| KanbanBoard.tsx                  | 5列と横スクロール           |
| KanbanColumn.tsx                 | 1列の抽出し・作成・カード       |
| KanbanCard.tsx                   | カード表示とstatus select |
| KanbanDetailAddForm.tsx          | 会社名・応募日の登録          |
| kanbanApi.test.ts                | 簡易登録APIの呼び出しテスト     |

### A.7 applications

| ファイル                                         | 役割                    |
|----------------------------------------------|-----------------------|
| features/applications/types.ts               | 応募一覧・詳細・入力型           |
| features/applications/api/applicationsApi.ts | 応募作成・詳細・更新・削除         |
| ApplicationCreatePanel.tsx                   | 求人から応募する応募画面。現在ルート未使用 |

### A.8 interviews

| ファイル                            | 役割                  |
|---------------------------------|---------------------|
| features/interviews/types.ts    | enum、API、フォーム型、日時変換 |
| api/interviewsApi.ts            | 応募別APIID            |
| components/InterviewSection.tsx | 面接State・API・通信中ID   |
| components/InterviewForm.tsx    | 面接入力と422表示          |
| components/InterviewList.tsx    | 一覧・編集・削除確認          |
| api/interviewsApi.test.ts       | URL、HTTPメソッド、JS断テスト |

### A.9 tasks

| ファイル                       | 役割                 |
|----------------------------|--------------------|
| features/tasks/types.ts    | 優先度、API、フォーム型、日時変換 |
| api/tasksApi.ts            | 応募別APIIDと完了更新      |
| components/TaskSection.tsx | タスクState・並び順・通信    |
| components/TaskForm.tsx    | タスク入力              |
| components/TaskList.tsx    | 完了切替・期間超過・削除確認     |
| api/tasksApi.test.ts       | API契約テスト           |

### A.10 notes

| ファイル                       | 役割             |
|----------------------------|----------------|
| features/notes/types.ts    | API・フォーム型      |
| api/notesApi.ts            | 応募別APIID       |
| components/NoteSection.tsx | メモState・通信     |
| components/NoteForm.tsx    | 本文入力・文字数・422表示 |
| components/NoteList.tsx    | 一覧・編集・削除確認     |
| api/notesApi.test.ts       | API契約テスト       |

### A.11 companies / jobPostings

現在のMVPルートでは画面から利用していないが、既存API用コードとして残っている。

| ファイル                              | 役割               |
|-----------------------------------|------------------|
| companies/types.ts                | 企業型              |
| companies/api/companiesApi.ts     | 企業一覧・登録          |
| CompanyCreatePanel.tsx            | 企業登録フォーム         |
| jobPostings/types.ts              | 求人一覧・詳細・フォーム型    |
| jobPostings/api/jobPostingsApi.ts | 求人APIID          |
| JobPostingCard.tsx                | 求人カード            |
| JobPostingForm.tsx                | 企業選択・作成を含む求人フォーム |

JobPostingCardのリンク先/jobs/:idは現在routerに存在しないため、現在のMVPからは使用されない。

### A.12 shared

| ファイル                                  | 役割                      |
|---------------------------------------|-------------------------|
| shared/api/client.ts                  | Axios instance、[ET、401] |
| shared/api/apiErrorHandler.ts         | unknownエラーを表示用へ変換       |
| shared/types/api.ts                   | 共通成功・エラー型               |
| shared/routes/ProtectRoute.tsx        | 認証必須ガード                 |
| shared/routes/GuestRoute.tsx          | 未認証専用ガード                |
| shared/components/AuthErrorScreen.tsx | 認証確認エラーと再試行             |
| shared/components/loadingScreen.tsx   | 全画面ローディング               |
| shared/components/contentLoading.tsx  | コンテンツ内ローディング            |
| shared/components/InlinedAlert.tsx    | 共通エラー表示                 |
| shared/utils/date.ts                  | 日付表示、今日の日付              |

### A.13 public / assets

| ファイル               | 役割             |
|--------------------|----------------|
| public/favicon.svg | index.htmlから使用 |
| public/icons.svg   | 静的アイコン資産       |
| assets/hero.svg    | 現在の主要画面では未使用   |
| assets/react.svg   | Vite初期資産。未使用   |
| assets/vite.svg    | Vite初期資産。未使用   |

## 付録B API対応表

| フロント関数                                   | HTTP   | URL                                              |
|------------------------------------------|--------|--------------------------------------------------|
| <code>authApi.signup</code>              | POST   | <code>/api/v1/auth</code>                        |
| <code>authApi.login</code>               | POST   | <code>/api/v1/auth/sign_in</code>                |
| <code>authApi.me</code>                  | GET    | <code>/api/v1/auth/me</code>                     |
| <code>authApi.logout</code>              | DELETE | <code>/api/v1/auth/sign_out</code>               |
| <code>kanbanApi.get</code>               | GET    | <code>/api/v1/kanban</code>                      |
| <code>kanbanApi.createApplication</code> | POST   | <code>/api/v1/kanban/applications</code>         |
| <code>kanbanApi.updateStatus</code>      | PATCH  | <code>/api/v1/applications/:id/status</code>     |
| <code>applicationsApi.create</code>      | POST   | <code>/api/v1/applications</code>                |
| <code>applicationsApi.get</code>         | GET    | <code>/api/v1/applications/:id</code>            |
| <code>applicationsApi.update</code>      | PATCH  | <code>/api/v1/applications/:id</code>            |
| <code>applicationsApi.destroy</code>     | DELETE | <code>/api/v1/applications/:id</code>            |
| <code>interviewsApi.list</code>          | GET    | <code>/api/v1/applications/:id/interviews</code> |
| <code>interviewsApi.create</code>        | POST   | <code>/api/v1/applications/:id/interviews</code> |
| <code>interviewsApi.update</code>        | PATCH  | <code>/api/v1/interviews/:id</code>              |
| <code>interviewsApi.destroy</code>       | DELETE | <code>/api/v1/interviews/:id</code>              |
| <code>tasksApi.list</code>               | GET    | <code>/api/v1/applications/:id/tasks</code>      |
| <code>tasksApi.create</code>             | POST   | <code>/api/v1/applications/:id/tasks</code>      |
| <code>tasksApi.update</code>             | PATCH  | <code>/api/v1/tasks/:id</code>                   |
| <code>tasksApi.destroy</code>            | DELETE | <code>/api/v1/tasks/:id</code>                   |
| <code>notesApi.list</code>               | GET    | <code>/api/v1/applications/:id/notes</code>      |
| <code>notesApi.create</code>             | POST   | <code>/api/v1/applications/:id/notes</code>      |
| <code>notesApi.update</code>             | PATCH  | <code>/api/v1/notes/:id</code>                   |
| <code>notesApi.destroy</code>            | DELETE | <code>/api/v1/notes/:id</code>                   |
| <code>companiesApi.list</code>           | GET    | <code>/api/v1/companies</code>                   |
| <code>companiesApi.create</code>         | POST   | <code>/api/v1/companies</code>                   |
| <code>jobPostingsApi.list</code>         | GET    | <code>/api/v1/job_postings</code>                |
| <code>jobPostingsApi.get</code>          | GET    | <code>/api/v1/job_postings/:id</code>            |
| <code>jobPostingsApi.create</code>       | POST   | <code>/api/v1/job_postings</code>                |
| <code>jobPostingsApi.update</code>       | PATCH  | <code>/api/v1/job_postings/:id</code>            |
| <code>jobPostingsApi.destroy</code>      | DELETE | <code>/api/v1/job_postings/:id</code>            |

## 付録C 学習用語集

| 用語                   | 意味                                   |
|----------------------|--------------------------------------|
| SPA                  | ページ全体を再読み込みせず画面を切り替えるWebアプリ          |
| Component            | UIを表す再利用可能な関数                        |
| JSX                  | JavaScript/TypeScript内でHTML風にUIを書く構文 |
| Props                | 親から子へ渡す値                             |
| State                | Componentが保持し、変更時に再描画される値            |
| Hook                 | useStateなど、関数Componentへ機能を追加する関数     |
| Context              | 複数階層で共通値を共有する仕組み                     |
| Provider             | Contextの値を子孫へ提供するComponent           |
| Interceptor          | HTTP通信の前後へ共通処理を差し込む機能                |
| JWT                  | 認証情報を表す署名付きトークン                      |
| Bearer               | Authorizationヘッダーでトークンを送る方式          |
| controlled component | inputの値をReact Stateで管理する方法           |
| cleanup              | Effect終了時に監視や処理を解除する関数               |
| immutable            | 元データを直接変更せず新しい値を作る考え方                |
| Union型               | 複数の候補値だけを許可する型                       |
| Generics             | 利用時に具体的な型を決める仕組み                     |
| mock                 | テストで本物の通信や処理を置き換える偽物                 |
| CRUD                 | Create、Read、Update、Delete            |
| 422                  | 入力内容がルールに合わない                        |
| 401                  | 認証が必要、またはJWTが無効                      |
| 404                  | 対象が存在しない、またはアクセスできない                 |
| 429                  | リクエスト回数が制限を超えた                       |

### 理解確認問題

1. `App.tsx`で`BrowserRouter`が一番外側にある理由は何か。
2. `user === null`だけで認証状態を判断しない理由は何か。
3. `/auth/me`が500のときJWTを削除しない理由は何か。
4. `Axios Request Interceptor`は何をしているか。
5. `KanbanData`を`Record`で表す利点は何か。
6. `sortCards`で`[...cards]`を使う理由は何か。
7. `Task`のAPI型とフォーム型が分かれている理由は何か。
8. `Set<number>`で通信中IDを管理する利点は何か。
9. 422エラーの`details`はどのように表示されるか。
10. 他ユーザーのデータ保護をReactだけに任せてはいけない理由は何か。

### 自分の言葉で説明する練習

次のテーマを1分ずつ説明できれば、プロジェクト理解はかなり深まっている。

- ・アプリ起動からカンバン表示まで
- ・ログインからJWT保存まで
- ・再読み込み時の認証復元
- ・カンバンのステータス変更
- ・タスク作成から一覧反映まで
- ・401と500の違い
- ・Contextを使う範囲
- ・Reduxを採用しなかった理由
- ・TypeScriptでAPIの安全性を高めた方法
- ・今後改善したいフロントエンド設計

## 最後に

JOBHUNTMANAGERのフロントエンドで最も重要な設計は、次の4点に整理できる。

1. 認証はAuthProviderとAxios Interceptorへ集約する
2. 画面、機能、API、型の責務を分ける
3. API結果をStateへ反映し、Reactの再描画でUIを更新する
4. 最終的な認証・認可・バリデーションはRails側でも保証する

コードを読むときは、個々の文法だけでなく、次の流れを追う。

```
ユーザー操作
→ Event Handler
→ APIモジュール
→ Axios Interceptor
→ Rails API
→ Response
→ State更新
→ React再描画
```

この一本の流れを各機能で説明できれば、React初心者の段階を越えて、アプリ全体の設計を理解し始めている。